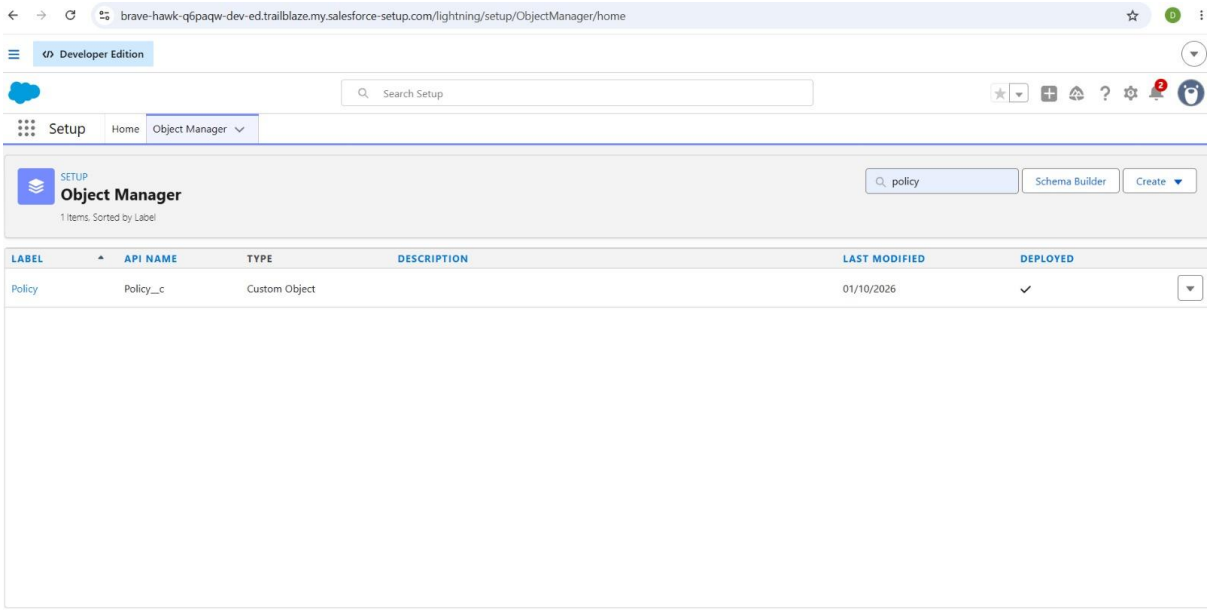
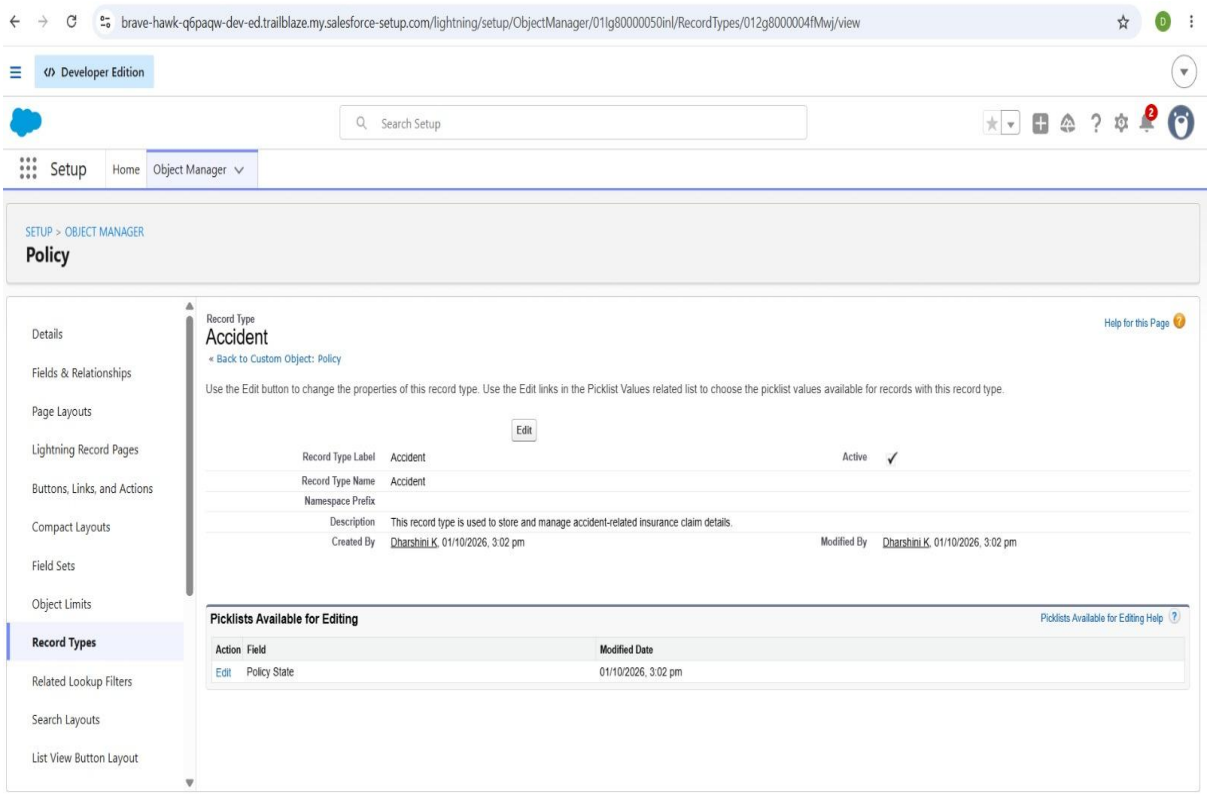
Milestone 1: Core Data Model & Policy Configuration

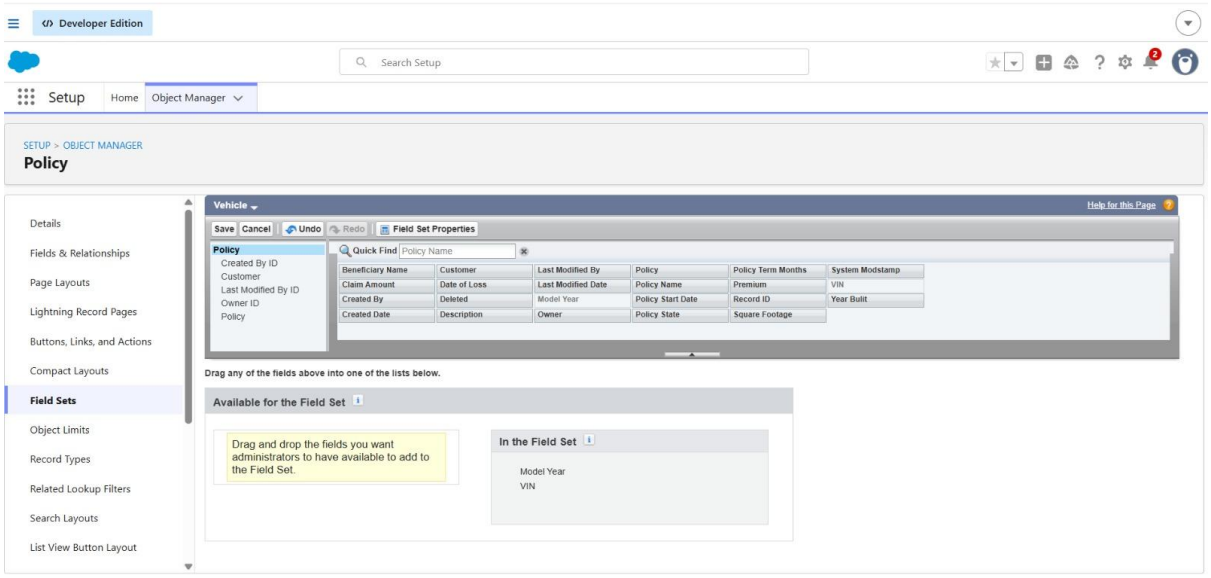
STEP 1 : Create the Policy custom object.

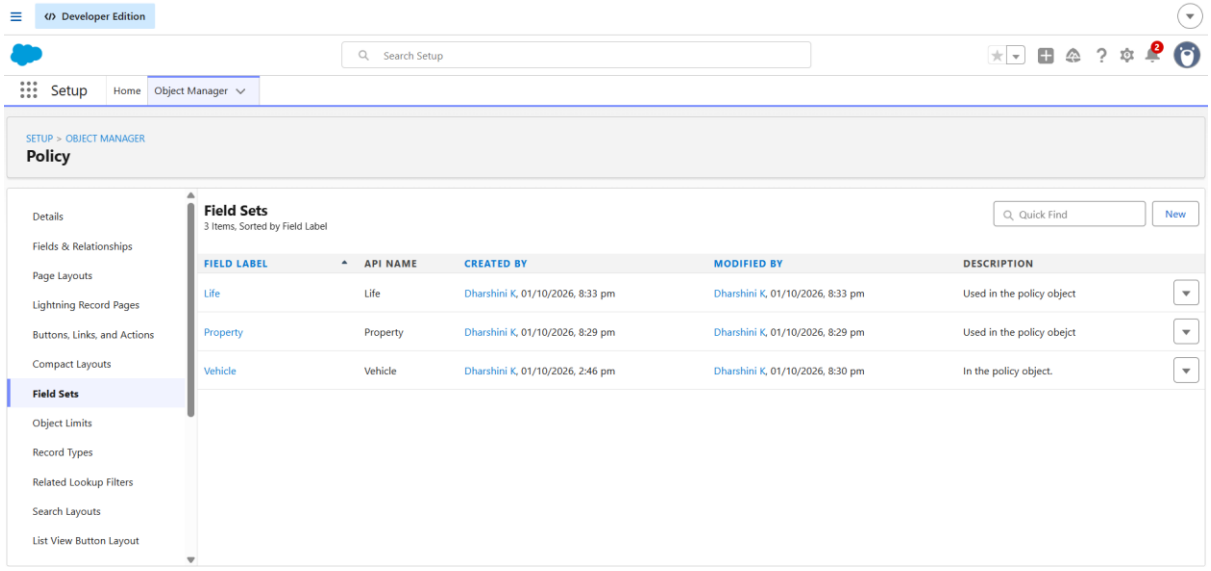


STEP 2 : Define Record Types on Policy: Auto, Property, and Life.

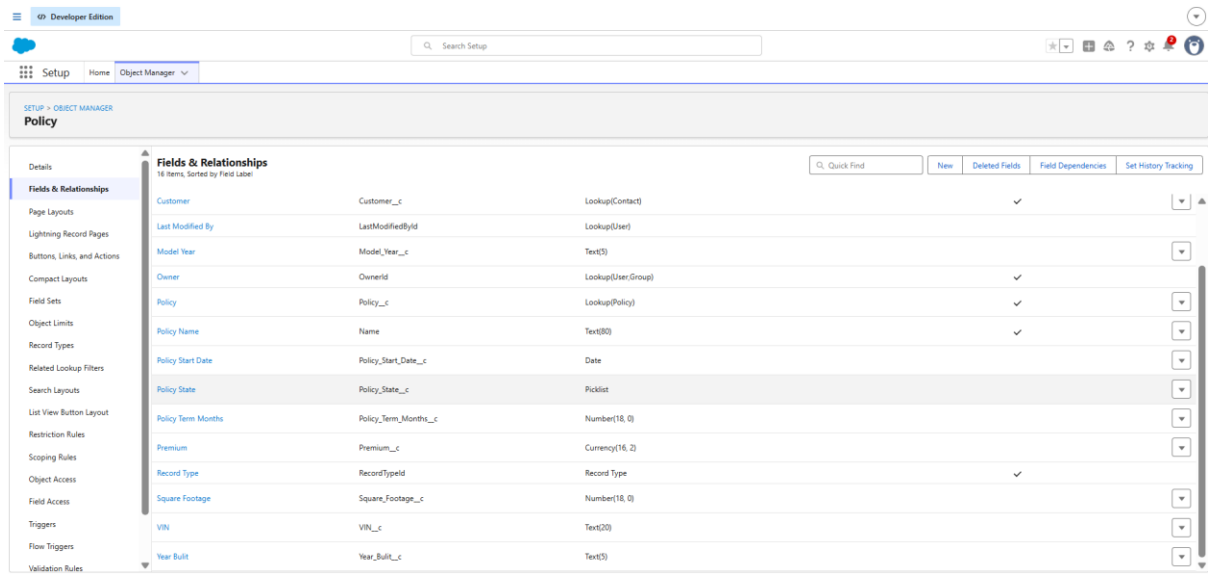


STEP 3 : Create Field Sets for policy-specific data

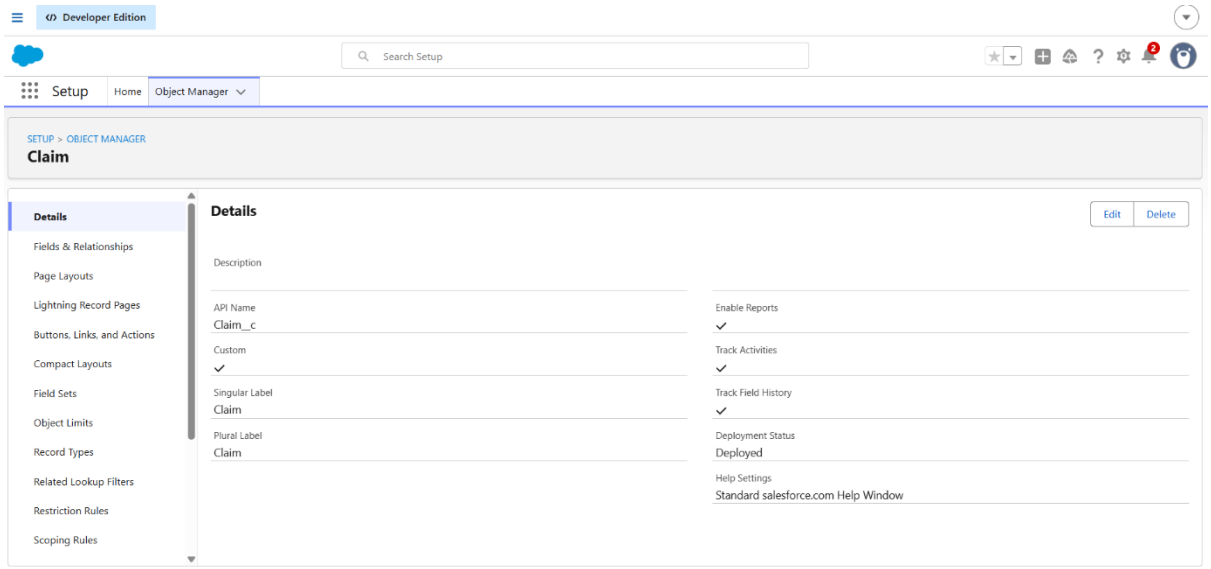




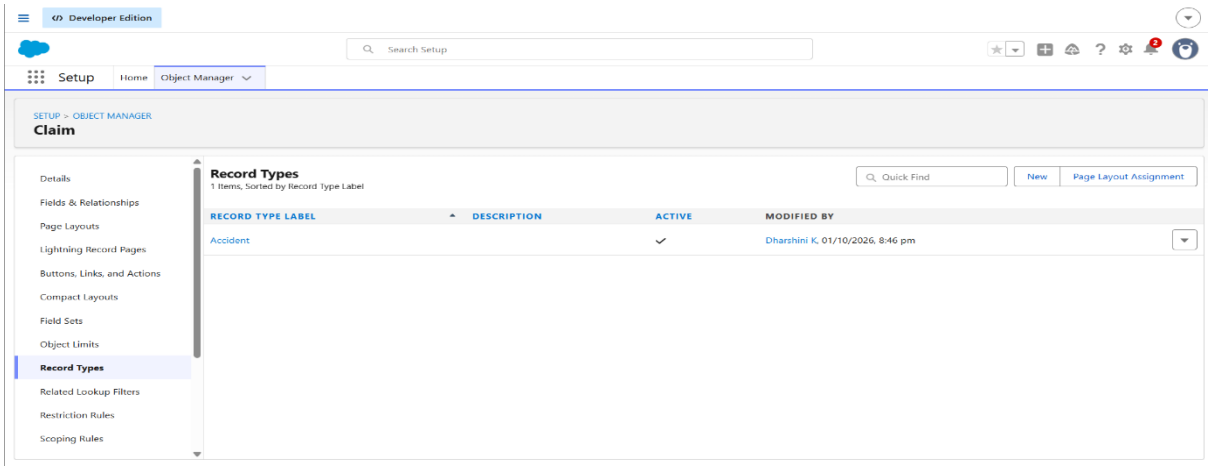
STEP 4 : Create Fields in the Policy Object

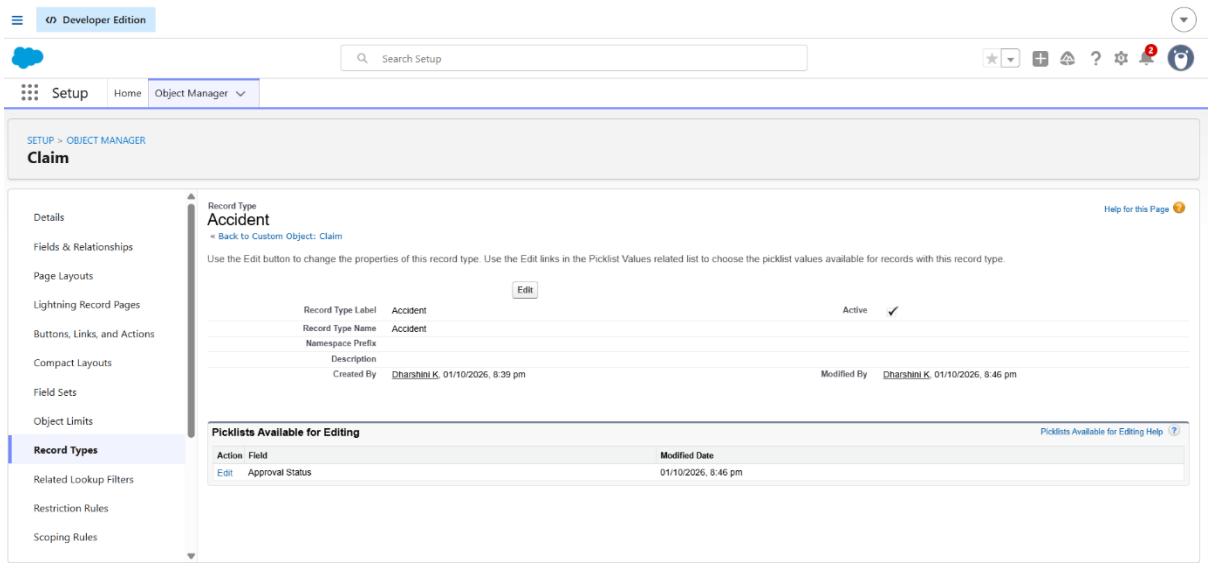


STEP 5 : Create Claim custom object

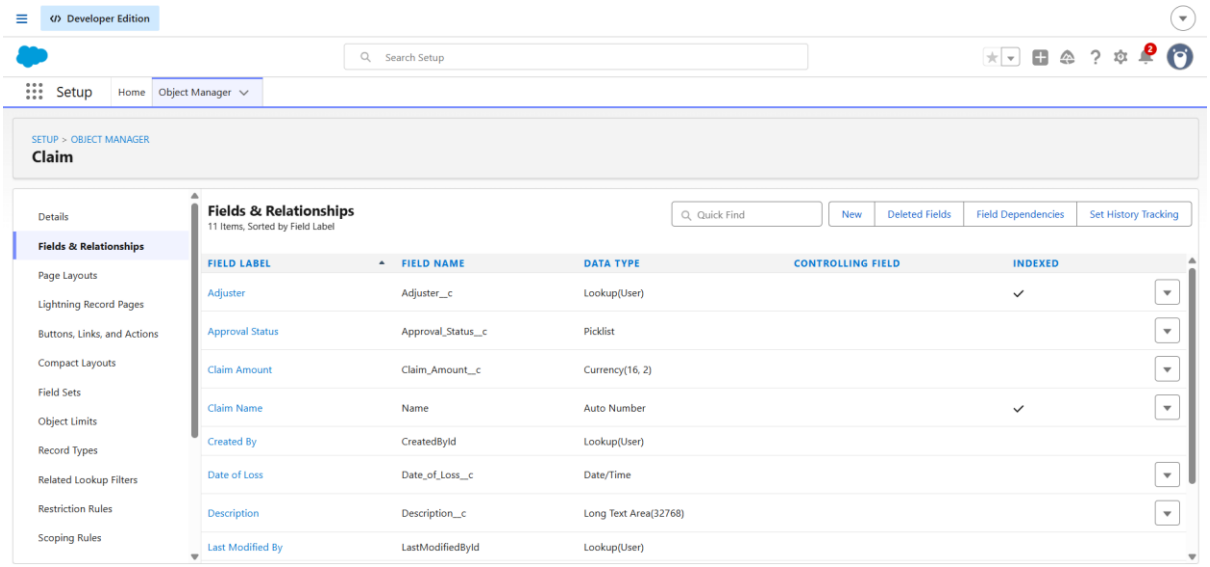


STEP 6 : Define Record Types on Claim Object

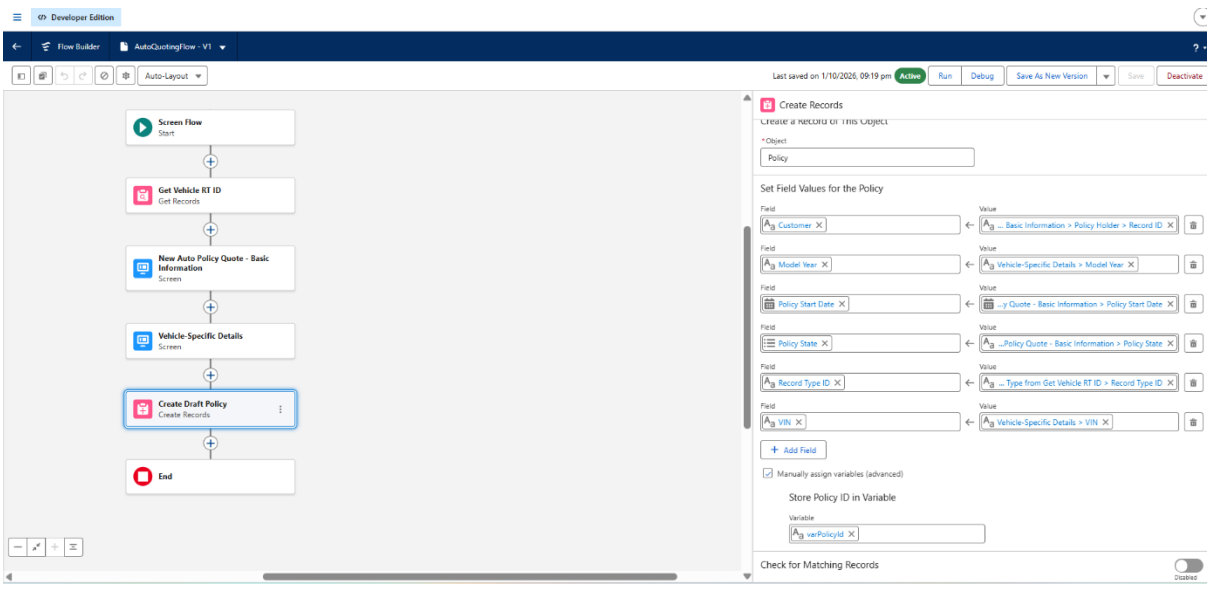




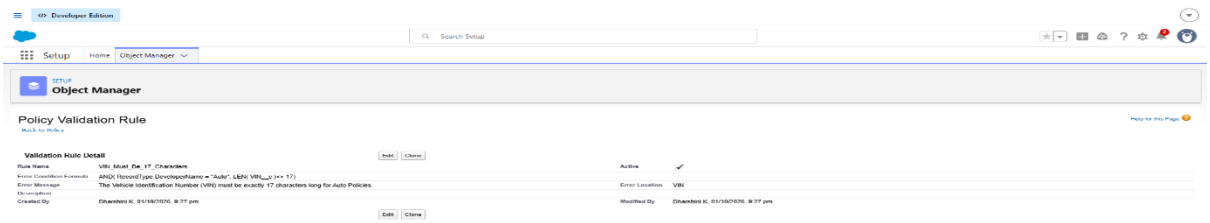
STEP 7 : Create Fields in the Policy Object



STEP 8 : Create an AutoQuoting Screen Flow

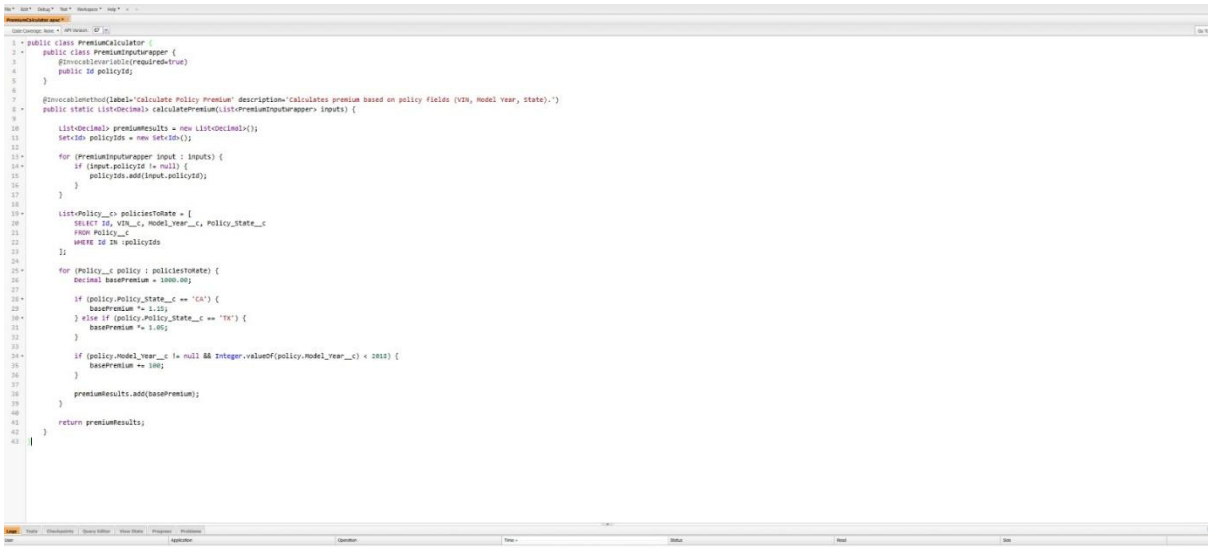


STEP 9 : Create Validation Rule on Policy Object



Milestone 2: Complex Policy Insurance & Claim Routing Automation

STEP 1 : Create an Apex Class with a method that accepts policy data



```
public class PremiumCalculator {
    public class PremiumInputWrapper {
        @InvocableVariable(required=true)
        public Id policyId;
    }

    @InvocableMethod(label='Calculate Policy Premium' description='Calculates premium based on policy fields (VIN, Model Year, State).')
    public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {

        List<Decimal> premiumResults = new List<Decimal>();
        Set<Id> policyIds = new Set<Id>();

        for (PremiumInputWrapper input : inputs) {
            if (input.policyId != null) {
                policyIds.add(input.policyId);
            }
        }

        List<Policy__c> policiesToRate = [
            SELECT Id, VIN__c, Model_Year__c, Policy_State__c
            FROM Policy__c
            WHERE Id IN :policyIds
        ];

        for (Policy__c policy : policiesToRate) {
            Decimal basePremium = 1000.00;

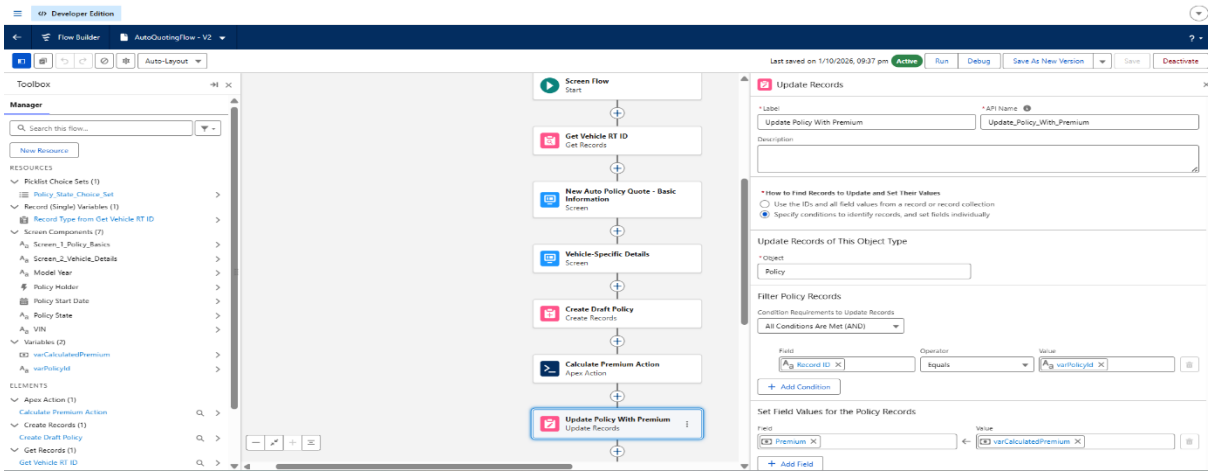
            if (policy.Policy_State__c == 'CA') {
                basePremium *= 1.15;
            } else if (policy.Policy_State__c == 'TX') {
                basePremium *= 1.05;
            }

            if (policy.Model_Year__c != null && Integer.valueOf(policy.Model_Year__c) < 2010) {
                basePremium += 100;
            }

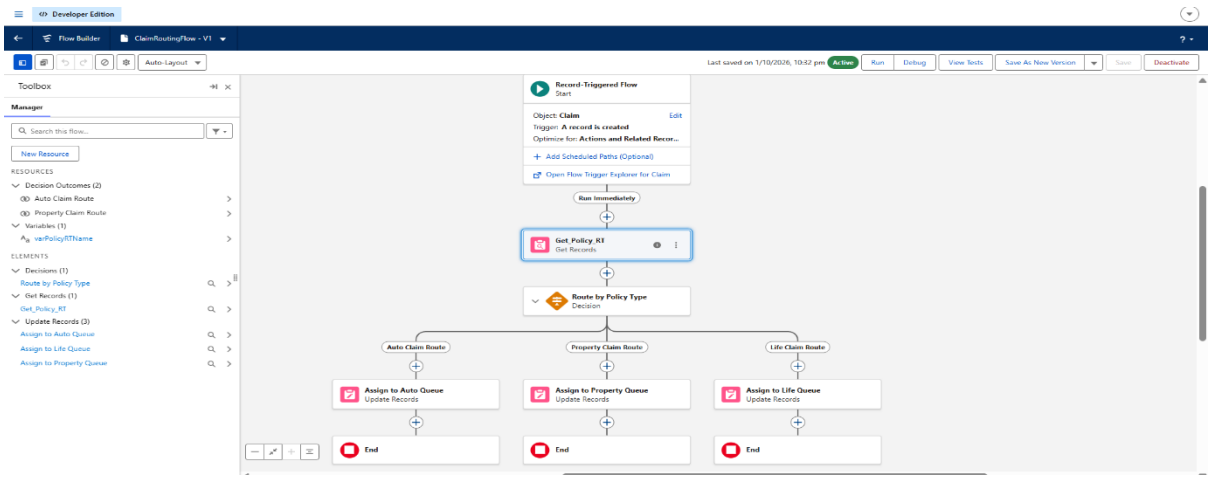
            premiumResults.add(basePremium);
        }

        return premiumResults;
    }
}
```

STEP 2 : Integrate this Apex action into the final step of the AutoQuotingFlow to update the draft Policy record

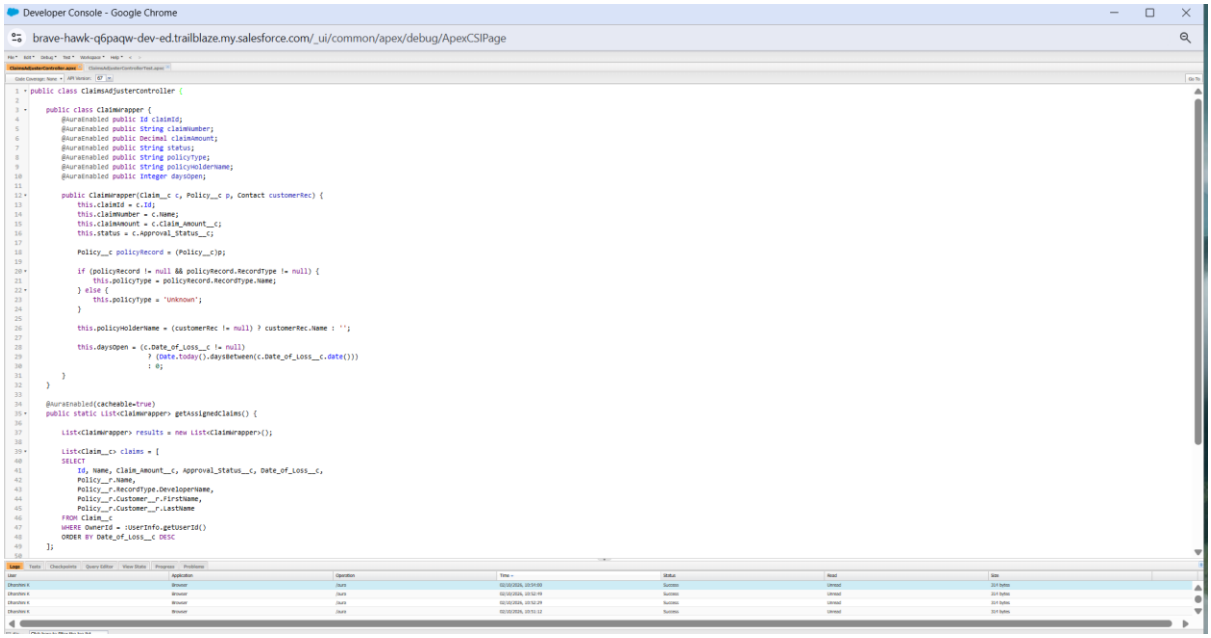


STEP 3 : Create a Record-Triggered Flow (After Save) on the Claim object.

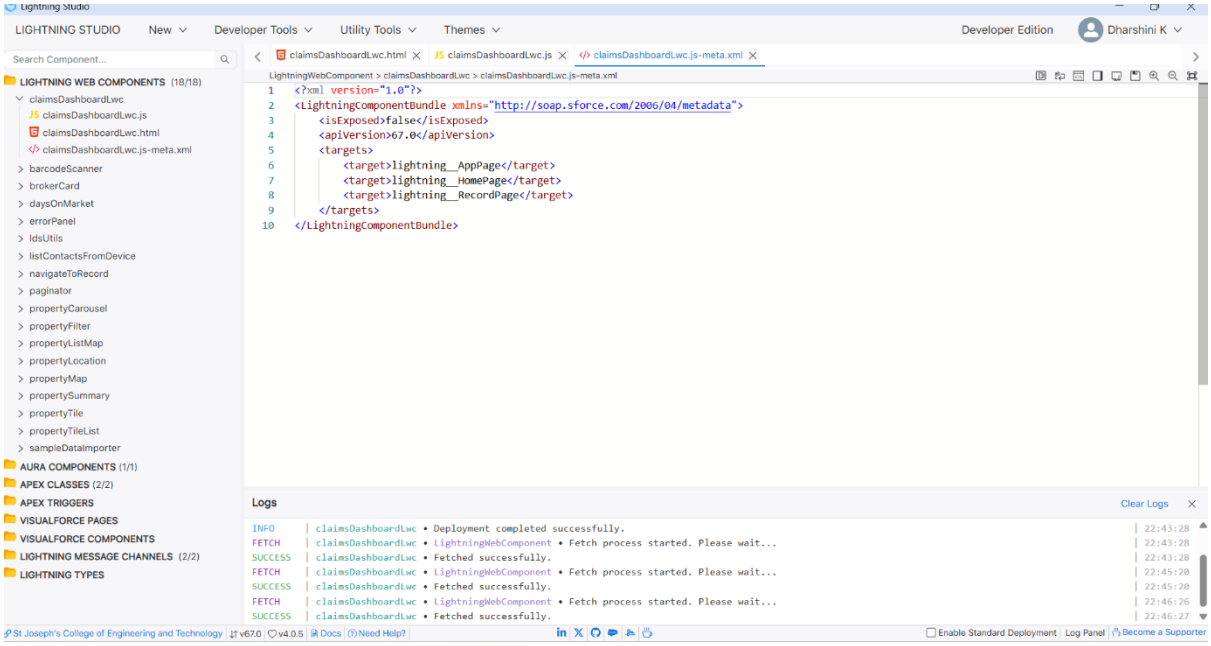


Milestone 3: Claims Adjuster LWC Dashboard Development

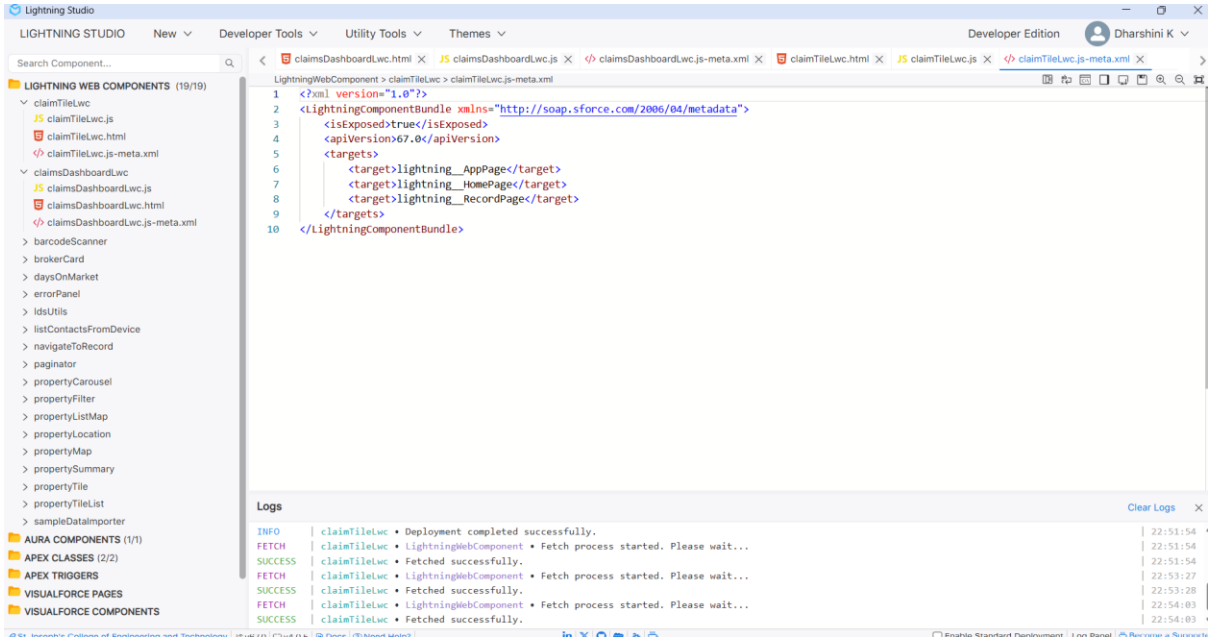
STEP 1 : Create a robust Apex class with a method using SOQL to query all Claims owned by the logged-in user, joining related Policy and Customer data. The method must return a custom Wrapper Class listing key metrics.



STEP 2 : Create a component that uses the Apex method via the @wire service and implements client-side data filtering

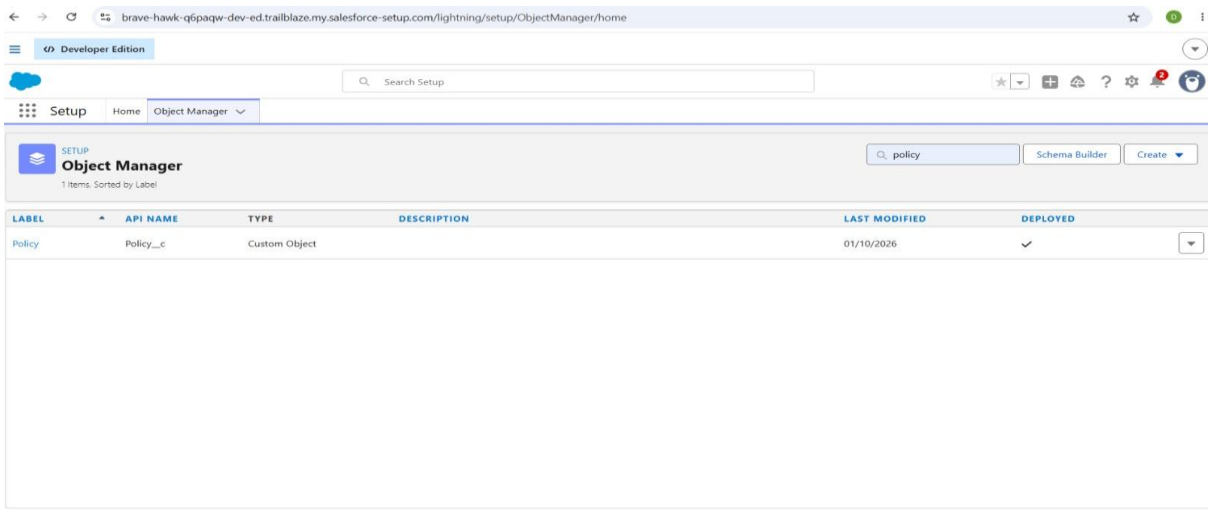


STEP 3 : Create a reusable component to display essential information for each claim

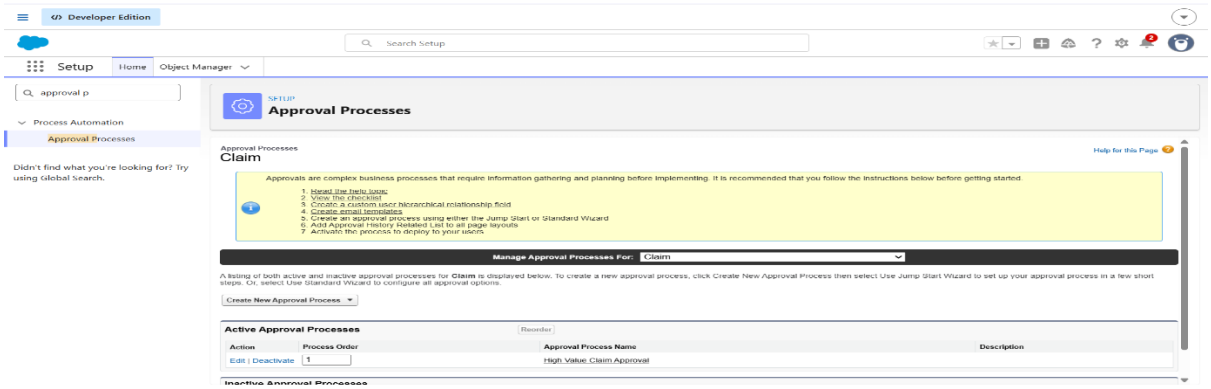


Milestone 4: Advanced Claim Processing, Security & Code Setup

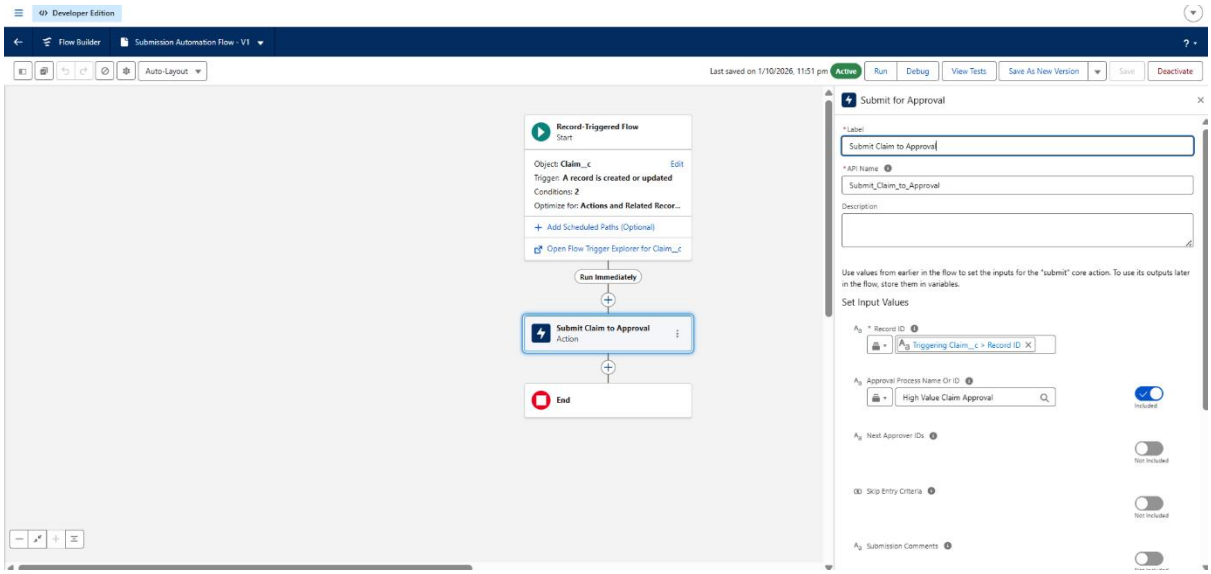
STEP 1 : Modify the Claim object to include an Approval Status field.



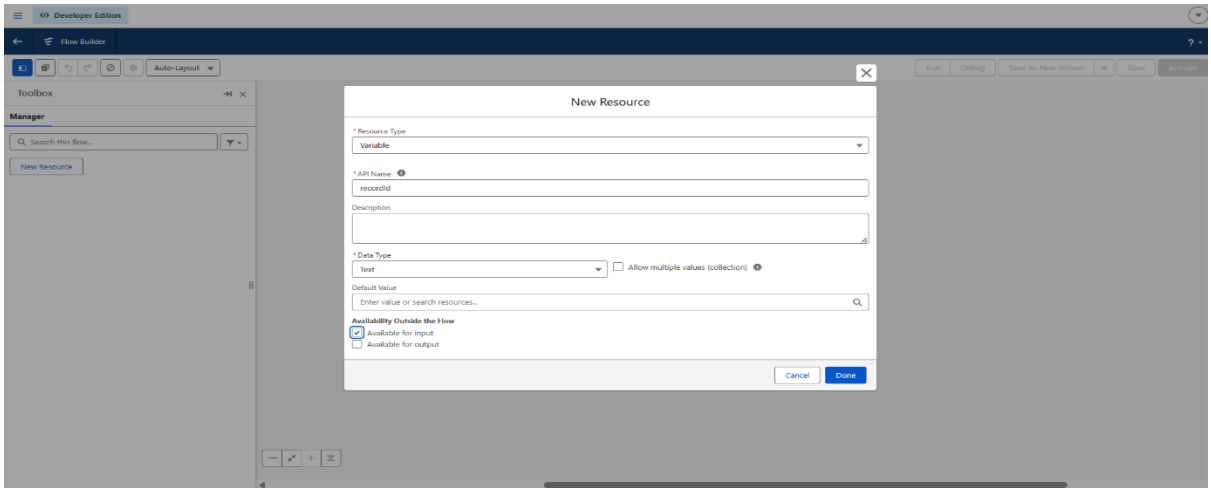
STEP 2 : Configure the Approval Process to route first to the Senior Adjuster, then to the Department Manager, using two sequential steps.

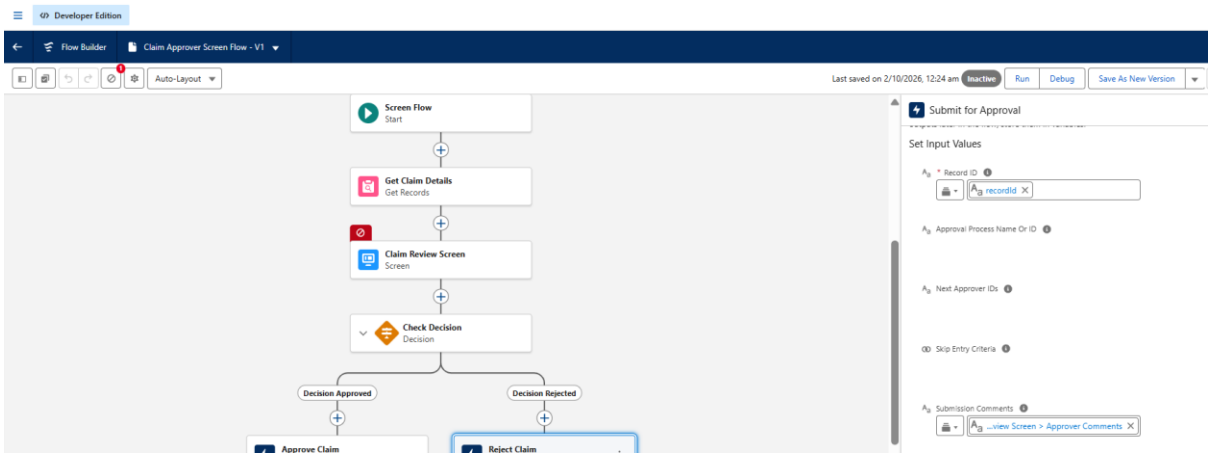


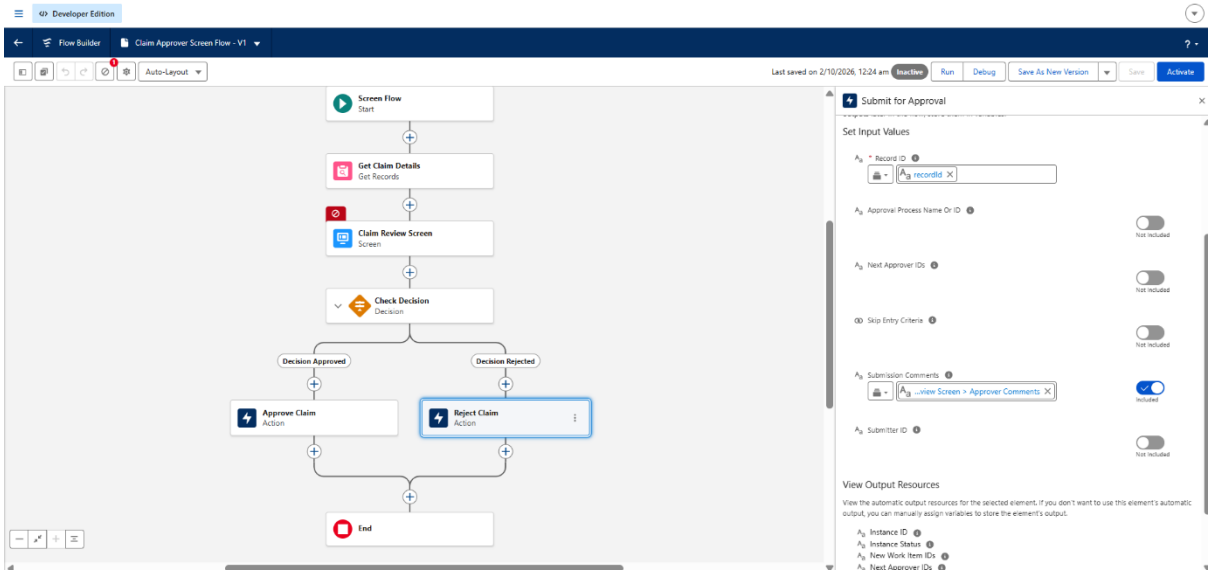
STEP 3 : Create a Record-Triggered Flow (After Save) that checks if the Claim Amount is $>\$50,000$ and submits the record to an Approval Process



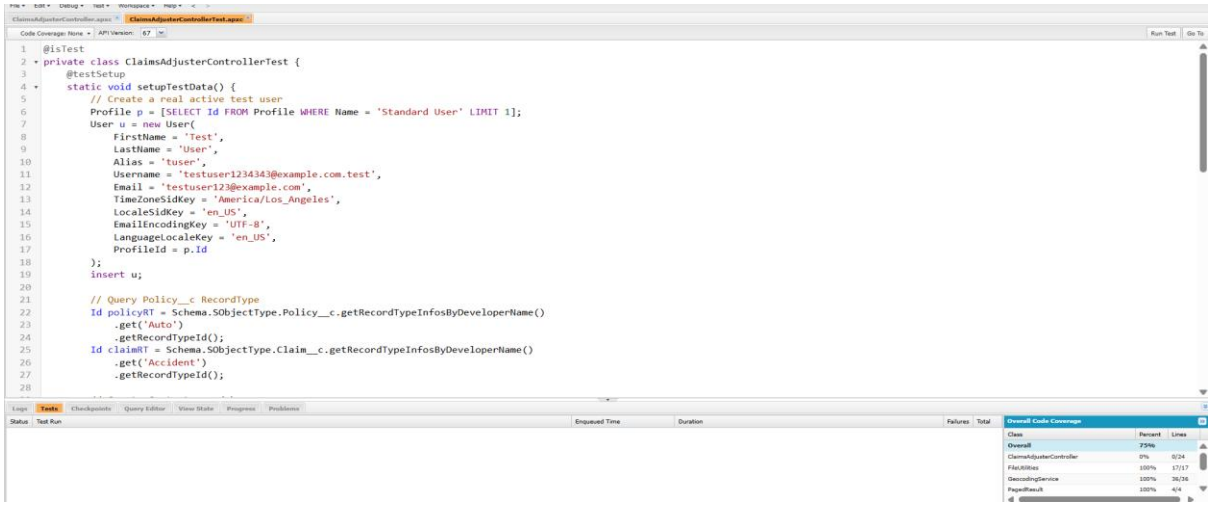
STEP 4 : Implement a Screen Flow for the approvers to quickly review details and approve/reject directly from the Claim record







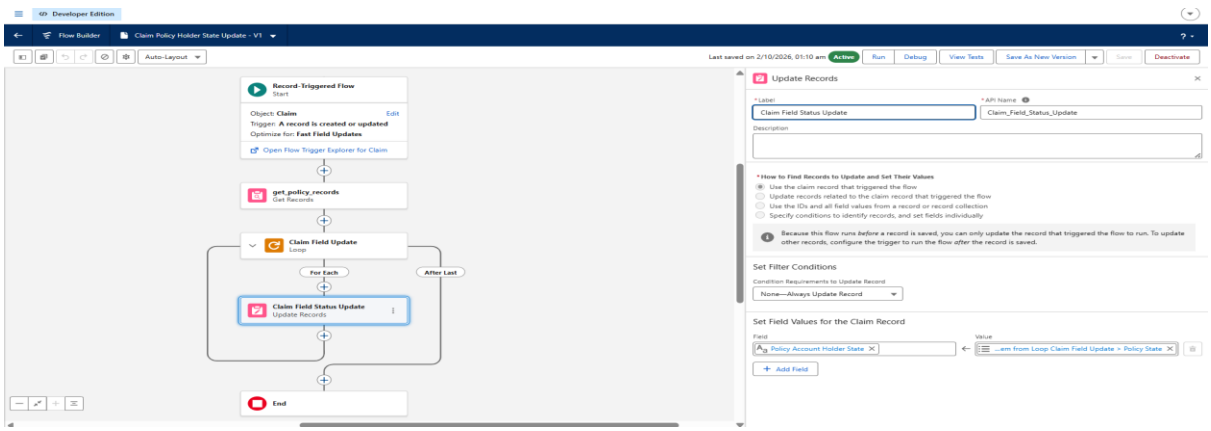
STEP 5 : Write an extensive Apex Test Class for the Claims Adjuster Controller with 95%+ code coverage. Include test methods for complex claim filtering and data retrieval logic



```apex
@isTest
private class ClaimsAdjusterControllerTest {
    @testSetup
    static void setupTestData() {
        // Create a real active test user
        Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1];
        User u = new User(
            FirstName = 'Test',
            LastName = 'User',
            Alias = 'tuser',
            Username = 'testuser1234343@example.com.test',
            Email = 'testuser123@example.com',
            TimeZoneSidKey = 'America/Los_Angeles',
            LocaleSidKey = 'en_US',
            EmailEncodingKey = 'UTF-8',
            LanguageLocaleKey = 'en_US',
            ProfileId = p.Id
        );
        insert u;

        // Query Policy__c RecordType
        Id policyRT = Schema.SObjectType.Policy__c.getRecordTypeInfosByDeveloperName()
            .get('Auto')
            .getRecordTypeId();
        Id claimRT = Schema.SObjectType.Claim__c.getRecordTypeInfosByDeveloperName()
            .get('Accident')
            .getRecordTypeId();
```

STEP 6 : Refine the Sharing Rules to ensure Adjusters only see claims in their assigned territory/state



STEP 7: Finalize all Permission Sets to differentiate access between Agents , Adjusters , and Managers

